

From Snakes to Crabs

A story about rewriting a critical service in Rust.



Dylan Martin

@dmarticus Tech lead, PostHog

BEFORE WE GET STARTED

Pop quiz.

01 How many people know what feature flags are?

Pop quiz.

01 How many people know what feature flags are?

02 How many people know Rust?

Pop quiz.

01 How many people know what feature flags are?

02 How many people know Rust?

03 How many people have managed a service under high load (>10k rps)?

WHO I AM

Dylan Martin

WHO I AM

Dylan Martin

Former tech lead, Feature Flags @ PostHog

WHO I AM

Dylan Martin

Former tech lead, Feature Flags @ PostHog

Member, OpenFeature Vendor Council

WHO I AM

Dylan Martin

Former tech lead, Feature Flags @ PostHog

Member, OpenFeature Vendor Council

Recommendations

Received (1)

Given (1)



Veronica (Pohls) Krey  · 1st

Director of Recruiting at Mercury

January 13, 2021, Veronica worked with Dylan but on different teams

A bit smart or whatever.

What a feature flag service does.

01 Request comes in

02 Service evaluates

03 Returns a decision

THE STAKES

Feature flags fail differently.

MOST SERVICES

Slow = bad UX for you.

FEATURE FLAGS

Slow = bad UX for your customers' customers.

STARTING POINT

Where we started.

500k

REQUESTS / MINUTE

300

DJANGO PODS

\$8.8k

/ MONTH IN COMPUTE

Projection: **10× growth** within a couple of years.

The rest of this talk, in three questions.

- | | | |
|----|----------------------|---|
| 01 | Choosing Rust | Why rewrite at all; and why Rust? |
| 02 | Architecture | The language-agnostic performance gains that came from changing the architecture. |
| 03 | Execution | How do you ship it without breaking production? |
-

SECTION 01

Choosing Rust.

Why we chose to rewrite, and how we picked the language.

Choosing Rust.

01 Flag logic trapped in SQL

Property matching and cohort joins ran inside the query. Cost grew with every flag, so we were CPU-bound even when the DB was fast.

Choosing Rust.

01 Flag logic trapped in SQL

Property matching and cohort joins ran inside the query. Cost grew with every flag, so we were CPU-bound even when the DB was fast.

02 Throughput capped per pod

Django's sync workers maxed out early. It took ~200 pods to serve, and it buckled under traffic spikes.

Choosing Rust.

01 Flag logic trapped in SQL

Property matching and cohort joins ran inside the query. Cost grew with every flag, so we were CPU-bound even when the DB was fast.

02 Throughput capped per pod

Django's sync workers maxed out early. It took ~200 pods to serve, and it buckled under traffic spikes.

03 Reliability we couldn't control

Timeouts lived in PgBouncer settings. When the DB slowed they failed outright, and the service went down. (Feb 2024.)

So: what does a replacement have to do?

The bottlenecks became our non-negotiables. The tiebreakers settled the rest.

So: what does a replacement have to do?

The bottlenecks became our non-negotiables. The tiebreakers settled the rest.

TABLE STAKES

≥1.5k rps per pod

Code-level timeout primitives

Production-ready ecosystem

TIEBREAKERS

Team can ship in it

FFI story to share a core across SDKs

How the three options measured up.

	Async Python FASTAPI	Node HYPEREXPRESS	Rust AXUM
Reuses existing logic	Yes (Python)	Yes (Node SDK)	No, from scratch
Team familiarity	High	High	Medium
Efficiency ceiling	Same problem, deferred	Higher	Highest
Framework risk	Low	High · 1.3k★ C++ binding	Low · Axum, 15k★
Prior prod proof at PostHog	—	No	Yes (capture-rs)
DEAL - BREAKER	CPU CEILING UNCHANGED	LOCKED TO A NICHE C++ BINDING	NONE

A note on Go.

Its advantages overlapped with Rust's.

Real concurrency, real timeouts, mature runtime.

It would have been new at PostHog too.

Same learning-curve tax as Rust, without the prior in-house proof.

No PoC to derisk it.

capture-rs gave Rust a shipped reference at our scale. Go didn't have one.

Greenfield, with no PoC either way? **Go is defensible.**

SECTION 02

Not all of the speedup came from Rust.

The rewrite helped a lot. So did changing what the service actually does.

Every flag condition ran inside one SQL query.

```
-- ONE query per request. Conditions for EVERY flag,  
-- evaluated inline by Postgres:  
SELECT person.id, person.properties,  
       (person.properties->>'email' LIKE '%@acme.com' AND  
        (person.properties->>'plan')::text = 'enterprise') AS flag_a,  
       (cohort.id = ANY($2)) AS flag_b,  
       -- ... one expression per flag, for every flag  
FROM person  
  LEFT JOIN cohortpeople ON cohortpeople.person_id = person.id  
  LEFT JOIN cohort ON cohort.id = cohortpeople.cohort_id  
WHERE person.team_id = $1 AND person.id = $3;
```

Why that didn't scale.

01 JSON property matching defeats indexes.

`properties->>'plan'` can't use a btree. Every flag check turns into a sequential scan.

Why that didn't scale.

01 JSON property matching defeats indexes.

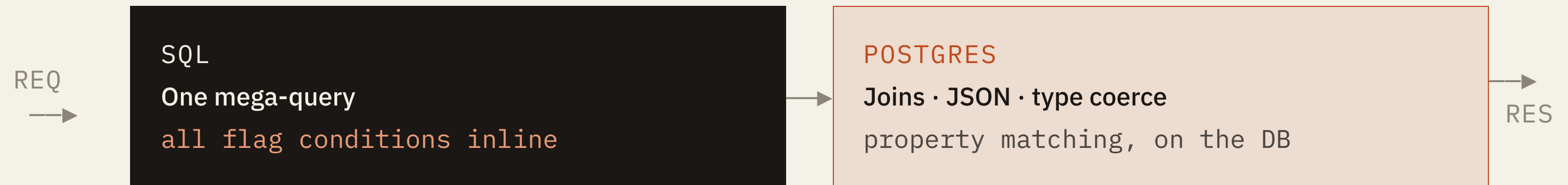
`properties->>'plan'` can't use a btree. Every flag check turns into a sequential scan.

02 ACID tax, paid 50 times per request.

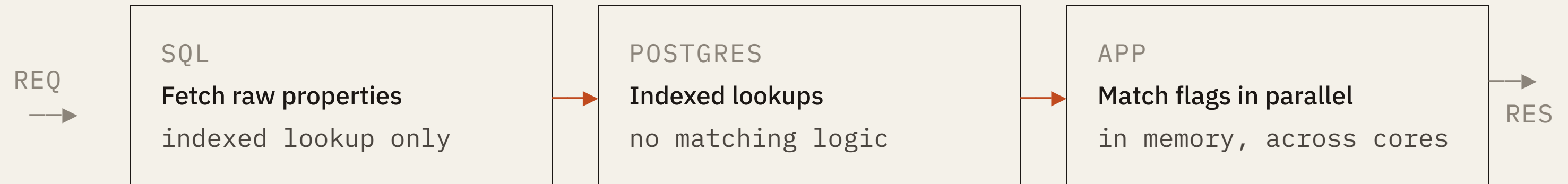
MVCC, snapshot isolation, WAL bookkeeping — per-condition overhead, for work you'll never roll back.

Fetch the raw properties. Match them in app memory.

BEFORE



AFTER



Stop re-querying cohorts that never change.

BEFORE

A fresh membership query, every request.

AFTER

Cache membership. Fetch it separately.

BENEFIT One of the heaviest joins, gone from every request.

Own the infrastructure Django outsourced.

Connection pooling

BEFORE

PgBouncer. A separate external hop.

AFTER

Native pool, in-process.

Concurrency

BEFORE

Django sync workers.

AFTER

Async runtime.

Timeouts

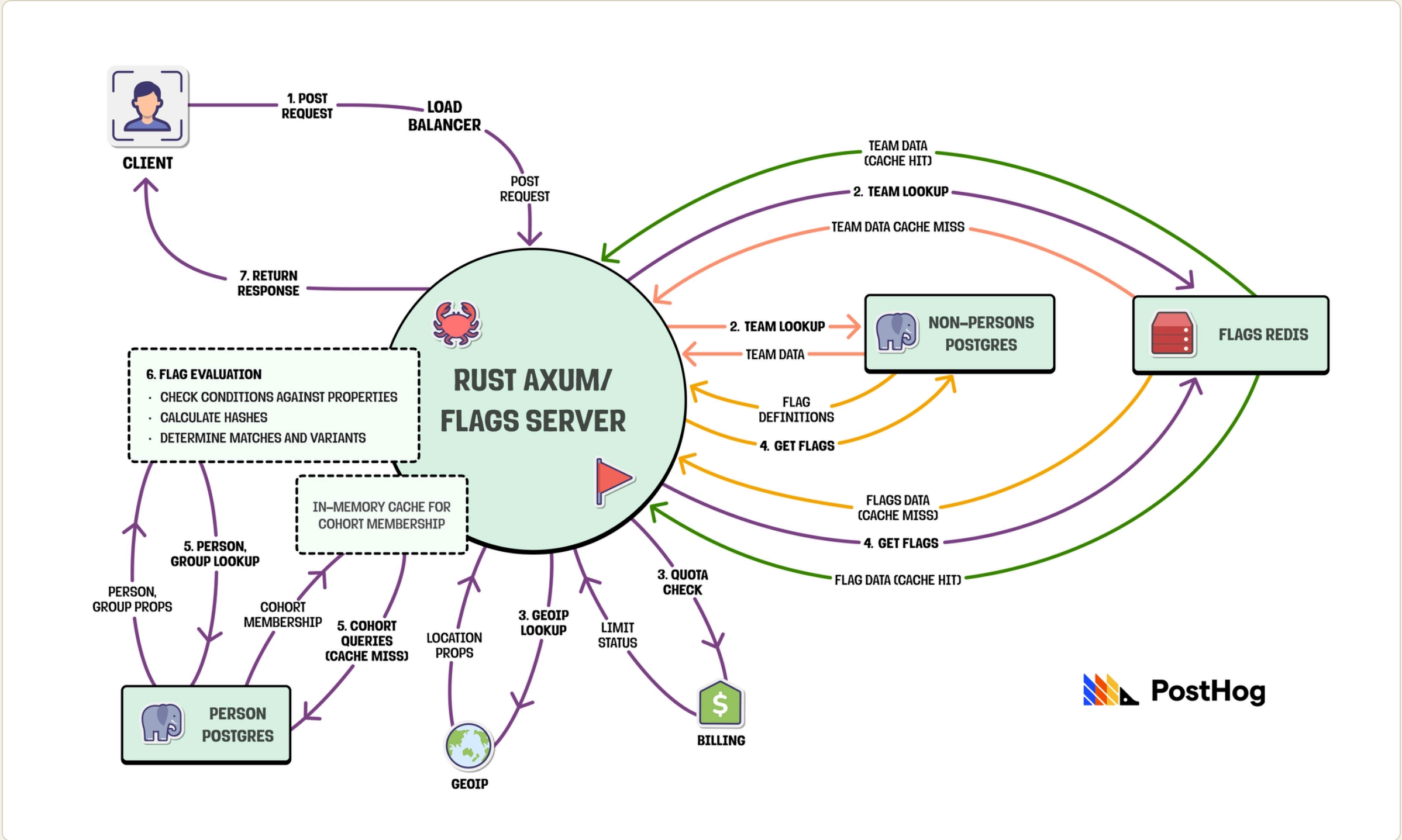
BEFORE

PgBouncer settings. Failed in the Feb-2024 outage.

AFTER

Code-level deadlines.

The new flags service, end to end.



SECTION 03

Shipping it without breaking production.

Solo. Three quarters. Zero feature freezes.

Solo. Three quarters. Boring.

1

ENGINEER ON THE REWRITE (ME)

<3

QUARTERS FROM KICKOFF TO CUTOVER

0

FEATURE FREEZES

Port what exists. Add what doesn't.

PORTED

Existing Python test suite

years of edge cases

ADDED

New integration tests

timeouts • lifecycle • requests

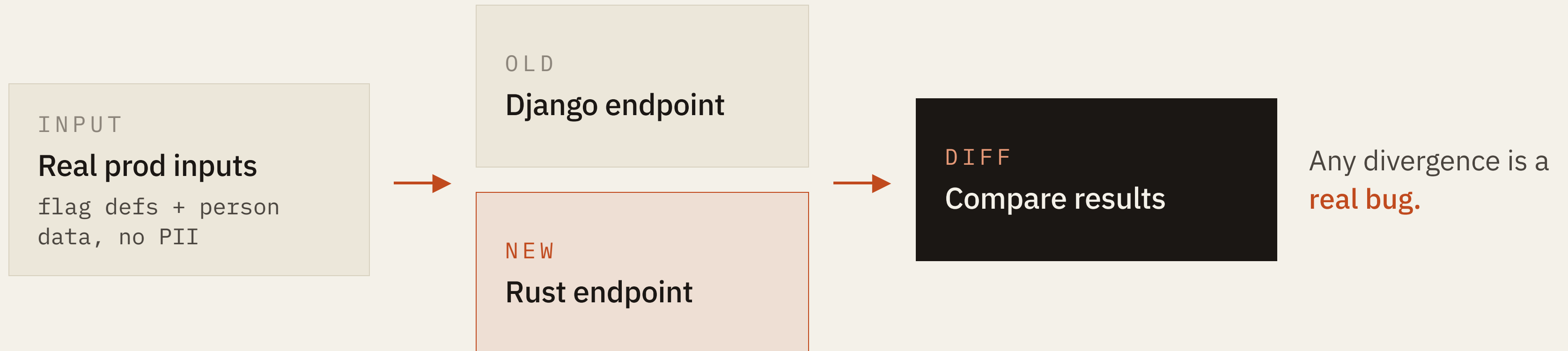


RESULT

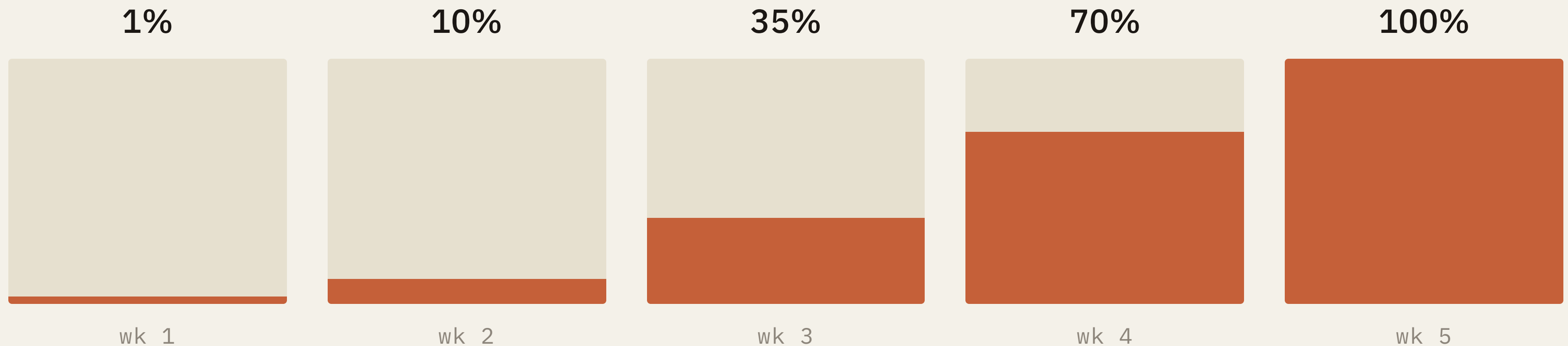
Rust test suite

nothing lost, new gaps covered

Make the new service prove it matches.



Roll out by API token. Slowly. Watch everything.



Share of traffic on Rust • hashed by API token

Every step doubled as a load test.

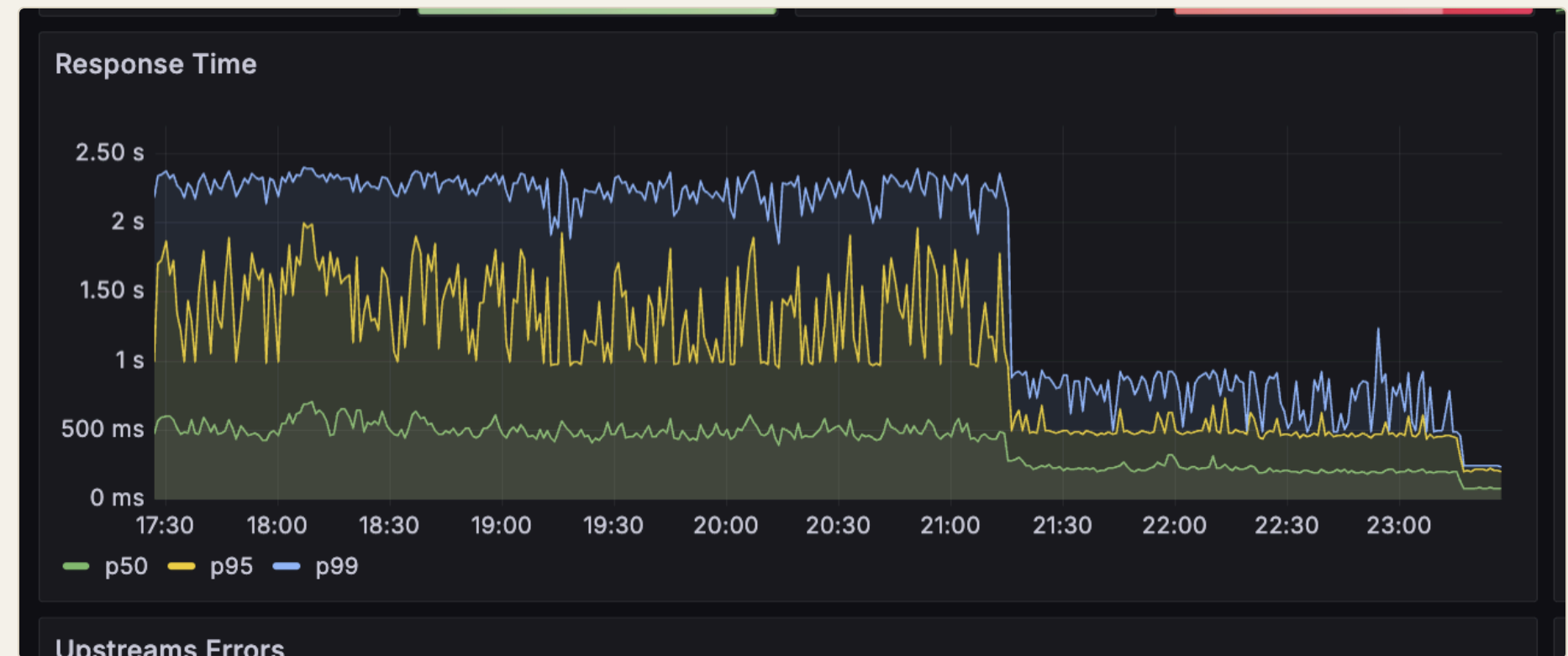
Latency.

P99

~~904ms~~

85ms

10.6× faster · 90.5% reduction



Response time across the rollout. Cutover $\approx$ 21:15.

p95 381 → 43 ms p90 160 → 31 ms p50 22 → 12 ms

Cost.

PODS	300	→	90	-70%
COMPUTE SPEND	\$8.8k	→	\$2.8k	-68%
TRAFFIC	500k rpm	→	500k rpm	same

Same traffic. Roughly one-third the compute.

Reliability.

THREE MONTHS POST-CUTOVER

Zero feature flag outages.

Then I wrote a blog post bragging about it. We had an outage the next week. That's another story.

What I'd do differently.

Property-based testing.

Load testing infrastructure.

Instrument the whole stack, not just the app.

Three takeaways.

01

The language wasn't the only lever.

02

Old tests are the best tool for preserving assumptions.

03

Boring is the goal.

POSTSCRIPT / SINCE OCTOBER

Here's now.

P99 TODAY

25ms



Live dashboard · May 2026.

END



Thank you.

Questions?

READ MORE

Part 1 · posthog.com/blog/even-faster-more-reliable-flags

Part 2 · posthog.com/blog/untangling-tokio-and-rayon

FIND ME

dylanamartin.com

github.com/dmarticus

linkedin.com/in/dmarticus